

第10章 排序

Faculty of Information and Control Engineering

Huanliang Sun

Email: sunhl@sjzu.edu.cn

Office: Wu(戊2-306) 24690666

第十章 内部排序

第一次课

10.1 概述

10.2 插入排序

练习与作业

第二次课

10.3 快速排序

10.4 选择排序

练习与作业

第三次课

10.5 归并排序

10.6 基数排序

10.7 多种排序方法的比较

练习与作业

归并排序：将两个或两个以上的有序表组合成一个新的有序表

—2-路归并排序过程

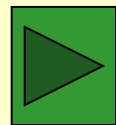
- 设初始序列含有 n 个记录，则可看成 n 个有序的子序列，每个子序列长度为1**
- 两两合并，得到 $\lfloor n/2 \rfloor$ 个长度为2或和1个长度为1的有序子序列**
- 再两两合并，如此重复，直至得到一个长度为 n 的有序序列**

初始关键字： [49] [38] [65] [97] [76] [13] [27]
 └──┘ └──┘ └──┘

一趟归并后： [38 49] [65 97] [13 76] [27]
 └────────┘ └────────┘

二趟归并后： [38 49 65 97] [13 27 76]
 └──────────┘

三趟归并后： [13 27 38 49 65 76 97]



❖ 算法描述

```
void Merge (RcdType SR[], RcdType TR[], int i, int m, int n){  
    // 将有序的SR[i..m]和SR[m+1..n]归并为有序的TR[i..n]  
    for (j=m+1, k=i; i<=m && j<=n; ++k){  
        // 将SR中记录按关键字从小到大复制到TR中  
        if (SR[i].key<=SR[j].key)  
            TR[k] = SR[i++];  
        else  
            TR[k] = SR[j++];  
    }  
    while (i<=m) TR[k++] = SR[i++];  
    // 将剩余的 SR[i..m] 复制到TR  
    while (j<=n) TR[k++] = SR[j++];  
    // 将剩余的 SR[j..n] 复制到TR  
} // Merge
```

```
void Msort ( RcdType SR[], RcdType TR1[], int s, int t ) {  
// 对SR[s..t]进行归并排序, 排序后的记录存入TR1[s..t]  
    if (s==t) TR1[s] = SR[s];  
    else {  
        m = (s+t)/2;  
        // 将 SR[s..t] 平分为 SR[s..m] 和 SR[m+1..t]  
        Msort (SR,TR2,s,m);  
        // 递归地将 SR[s..m] 归并为有序的 TR2[s..m]  
        Msort (SR,TR2,m+1, t);  
        // 递归地将SR[m+1..t]归并为有序的TR2[m+1..t]  
        Merge (TR2,TR1,s,m,t);  
        // 将TR2[s..m]和TR2[m+1..t] 归并到 TR1[s..t]  
    } // else  
} // Msort
```

```
void MergeSort (SqList &L){  
    // 对顺序表L作2-路归并排序  
    MSort(L.r, L.r, 1, L.length);  
} // MergeSort
```

算法评价:

- **时间复杂度:** 简单分析2-路归并排序的时间复杂度, 对一个长度为 n 的记录序列需进行 $\log_2 n$ 趟的归并, 而每一趟需进行关键字比较和记录移动的时间列和 n 成正比。

所以有: $T(n) = O(n \log_2 n)$

- **空间复杂度:** $S(n) = O(n)$

返回

练习

已知序列 {503, 87, 512, 61, 908, 170, 897, 275, 653, 462} 采用2路归并排序法对该序列做升序排序, 给出每一趟的结果。

Experiment 10-4 Merge Sorting

一、基数排序 借助多关键字排序

定义：假设有 n 个记录的序列： $\{R_1, R_2, \dots, R_n\}$ ，每个记录 R_i 中含有 d 个关键字 $(K_i^0, K_i^1, \dots, K_i^{d-1})$ ，则称上述记录序列**对关键字 $(K_i^0, K_i^1, \dots, K_i^{d-1})$ 有序**是指：对于序列中任意两个记录 R_i 和 $R_j (1 \leq i < j \leq n)$ 都**满足下列序关系**： $(K_i^0, K_i^1, \dots, K_i^{d-1}) < (K_j^0, K_j^1, \dots, K_j^{d-1})$ ，其中 K^0 称为**“最主”位关键字**， K^{d-1} 称为**“最次”位关键字**

对52张扑克牌按以下次序排序：

$\clubsuit 2 < \clubsuit 3 < \dots < \clubsuit A < \diamond 2 < \diamond 3 < \dots < \diamond A < \heartsuit 2 < \heartsuit 3 < \dots < \heartsuit A < \spadesuit 2 < \spadesuit 3 < \dots < \spadesuit A$

两个关键字：花色 ($\clubsuit < \diamond < \heartsuit < \spadesuit$)

面值 ($2 < 3 < \dots < A$)

并且“花色”地位高于“面值”

❖ 多关键字排序方法

➤ 最高位优先法 (MSD)

1. 先对最高位关键字 k^0 (如花色) 排序, 将序列分成若干子序列, 每个子序列有相同的 k^0 值;
2. 然后让每个子序列对次关键字 k^1 (如面值) 排序, 又分成若干更小的子序列;
3. 依次重复, 直至就每个子序列对最低位关键字 k^{d-1} 排序;
4. 最后将所有子序列依次连接在一起成为一个有序序列.

➤ 最低位优先法(LSD): 从最低位关键字 k^{d-1} 起进行排序, 然后再对高一位的关键字排序, 依次重复, 直至对最高位关键字 k^0 排序后, 便成为一个有序序列

➤ MSD与LSD不同特点:

- 按MSD排序, 必须将序列逐层分割成若干子序列, 然后对各子序列分别排序
- 按LSD排序, 不必分成子序列, 对每个关键字都是整个序列参加排序; 并且可不通过关键字比较, 而通过若干次分配与收集实现排序

例如:学生记录含三个关键字:系别 k^0 、班号 k^1 和班内的序列号 k^2 ，其中以系别为最主位关键字。LSD的排序过程如下:

无序序列	3,2,30	1,2,15	3,1,20	2,3,18	2,1,20
对 K^2 排序	1,2,15	2,3,18	3,1,20	2,1,20	3,2,30
对 K^1 排序	3,1,20	2,1,20	1,2,15	3,2,30	2,3,18
对 K^0 排序	1,2,15	2,1,20	2,3,18	3,1,20	3,2,30

按位排序必须是稳定的

二、链式基数排序

- 基数排序：借助“分配”和“收集”对单逻辑关键字进行排序的一种方法。
- 对于数字型或字符型的单关键字，看成是由多个数位或多个字符构成的多关键字，采用这种“分配-收集”的办法进行排序，数位或字符的取值范围称作基数。
- 链式基数排序：用链表作存储结构的基数排序

例如：对下列这组关键字

{209, 386, 768, 185, 247, 606, 230, 834, 539 }

首先按其“个位数”取值分别为0, 1, ..., 9“分配”成10组，之后按从0至9的顺序将它们“收集”在一起；

然后按其“十位数”取值分别为0, 1, ..., 9“分配”成10组，之后再按从0至9的顺序将它们“收集”在一起；

最后按其“百位数”重复一遍上述操作，便可得到这组关键字的有序序列。

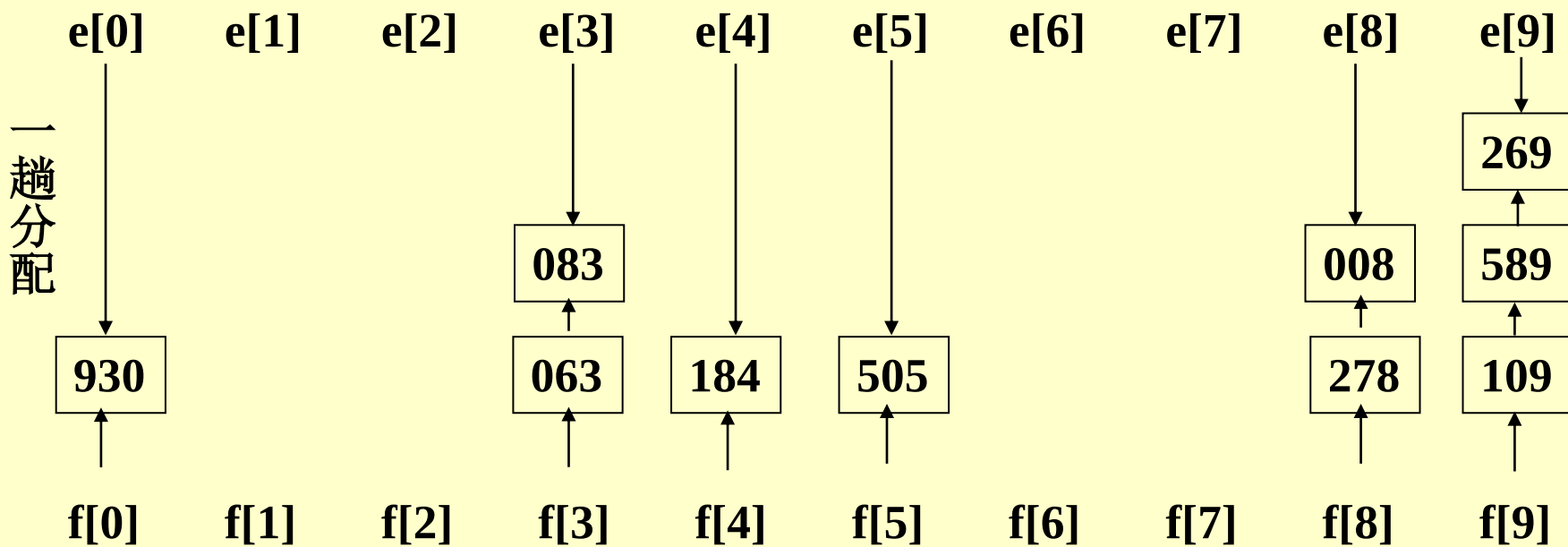
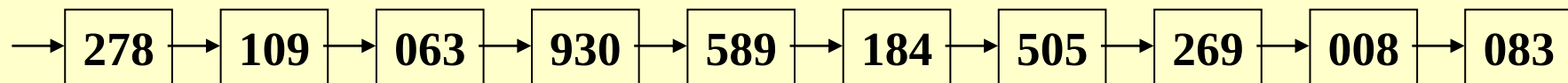
– 链式基数排序步骤

前提：待排序记录以指针相链，构成一个链表或静态链表；

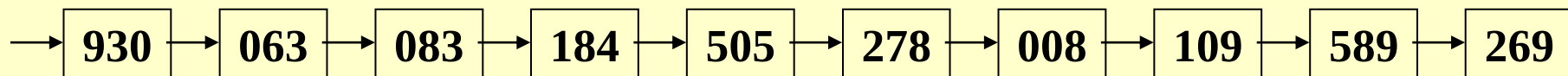
- 按照关键字的取值范围，设置队列，例如，对多位整型关键字，设置10个队列， $f[i]$ 和 $e[i]$ 分别为第 i 个队列的头指针和尾指针
- 第一趟分配对最低位关键字（个位）进行，改变记录的指针值，将链表中记录分配至10个链队列中，每个队列记录的关键字的个位相同
- 第一趟收集是改变所有非空队列的队尾记录的指针域，令其指向下一个非空队列的队头记录，重新将10个队列链成一个链表
- 重复上述两步，进行第二趟、第三趟分配和收集，分别对十位、百位进行，最后得到一个有序序列



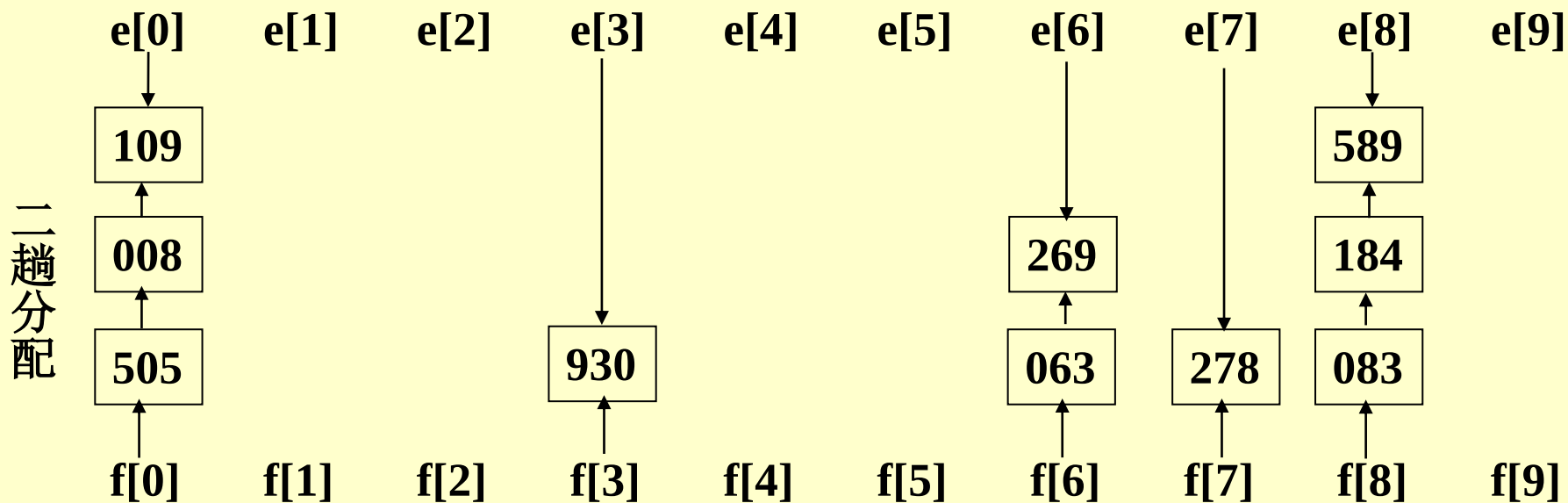
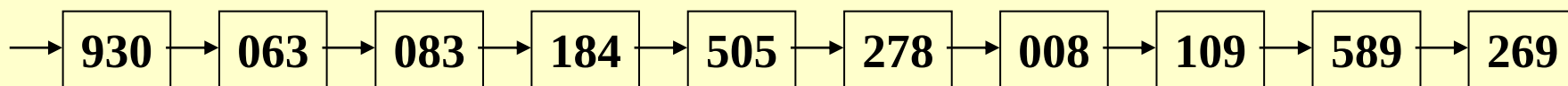
例 初始状态:



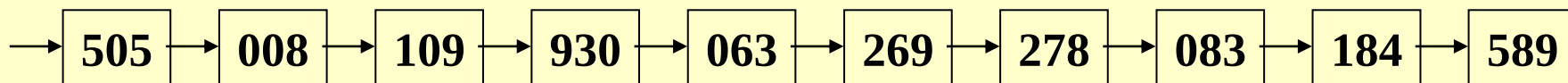
一趟收集:



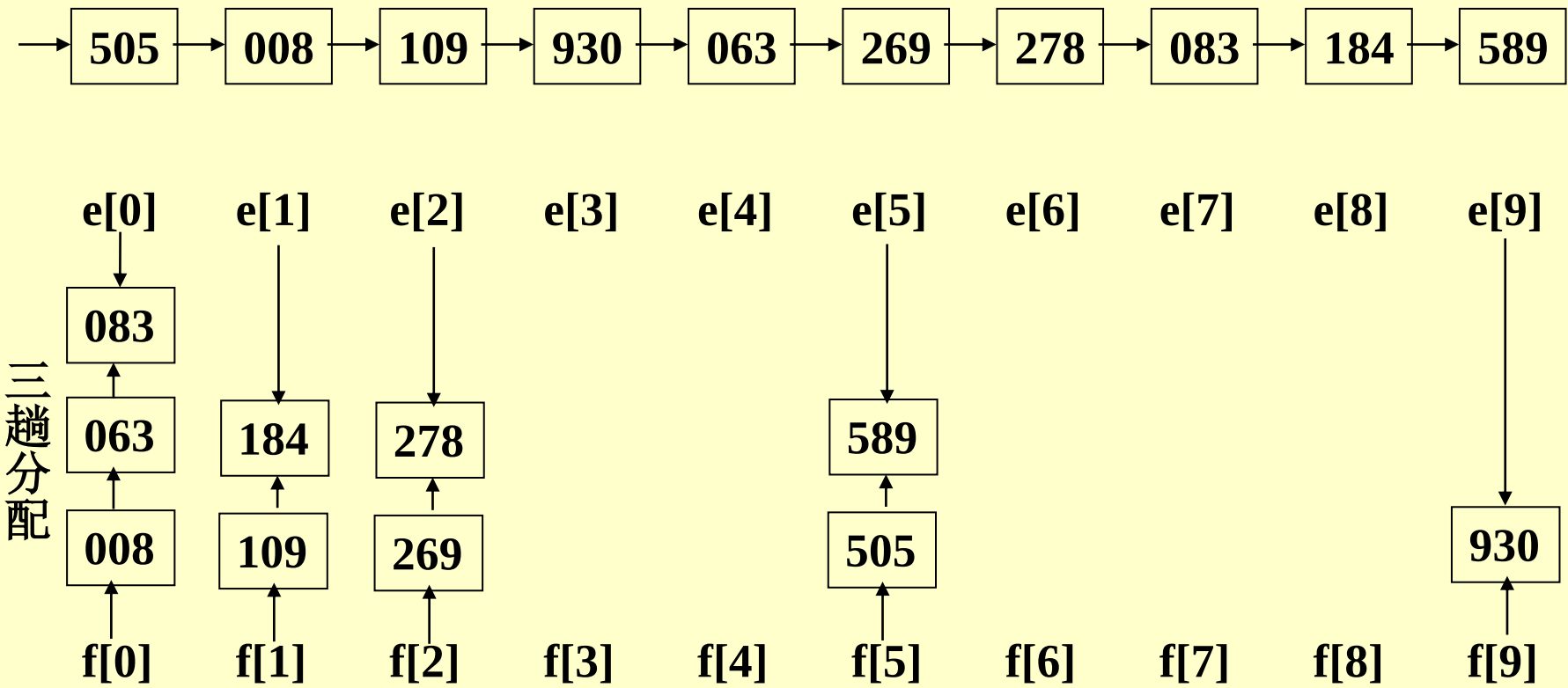
一趟收集:



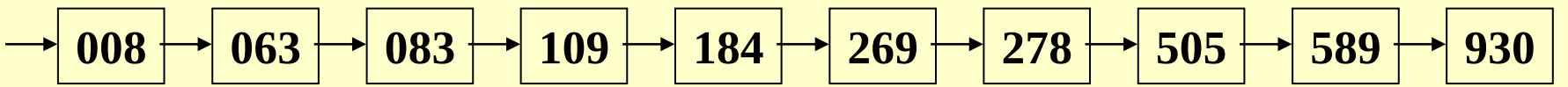
二趟收集:



二趟收集:



三趟收集:



在描述基数排序的算法之前，尚需重新定义记录和顺序表类型

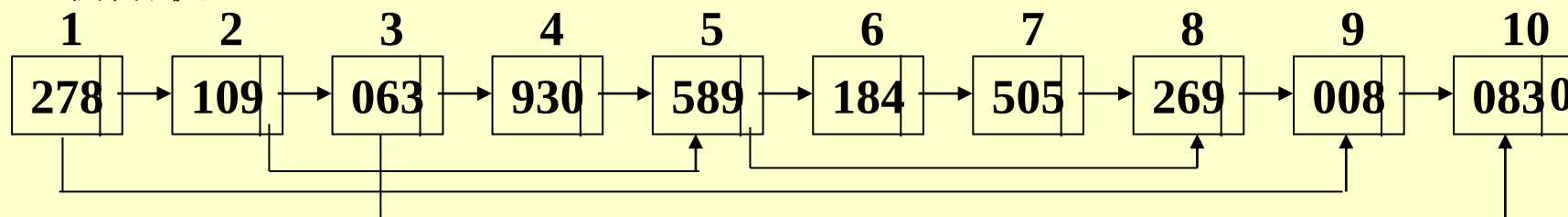
```
#define MAX_NUM_OF_KEY = 8;  
// 关键字项数的最大值，暂定为8  
#define RADIX = 10; // 关键字基数，此为十进制整数的基数  
#define MZX_SPACE 10000  
typedef struct {  
    KeysType keys[MAX_NUM_OF_KEY]; // 关键字  
    InfoType otheritems; // 其它数据项  
    int next;  
} SLCell; // 基数排序中的记录类型  
typedef struct {  
    SLCell r[MAX_SPACE];  
    int keynum; // 顺序表长度  
    int recnum; // 关键字位数  
} SLList; // 顺序表类型  
typedef int ArrayType[RADIX];
```

```
void Distribute (SLCell &r, int i, ArrType &f, Arrtype &e){  
    // r中记录已按(R.keys[0],...,R.keys[i-1])有序, 本算法按  
    // R.keys[i] 建立 RADIX 个子表, 使同一子表中记录的  
    // keys[i] 相同。 f[0..RADIX-1] 和e[0..RADIX-1] 分别指向  
    // 各子表中第一个和最后一个记录  
    for (j=0; j<Radix; ++j) f[j] = 0; // 各子表初始化为空表  
    for (p=r[0].next; p; p=r[p].next) {  
        j = ord(r[p].keys[i]); //取得低位关键字在记录中的下标  
        // ord 将记录中的关键字keys[k]映射到[0..RADIX-1]中  
        if ( !f[j] ) f[j] = p; //若第j个静态链表为空则头指针直接指向  
        else r[e[j]].next= p; //若第j个静态链表不为空插入子表  
        e[j] = p; //更改尾指针的指向  
    } // for  
} // Distribute
```

```
void Collect(SLList &r,int i,ArrType f,ArrType e)
{//收集算法。与分配算法相对
    int j,t;
    for(j=0;!f[j];j++);//找到第一个非空子表
    r.r[0].next=f[j];t=e[j];//r.r[0]指向第一个非空子表的第一个结点
    while(j<RADIX)
    {
        for(j++;j<RADIX-1&&!f[j];j++);//找到下一个非空子表
        if(f[j]) {r.r[t].next=f[j];t=e[j];}
    }
    r.r[t].next=0;//收集完成后最后一个结点的next指针指向0
}
```

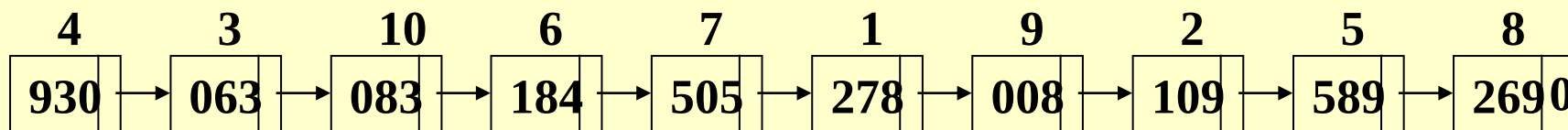
```
void RadixSort(SLList &L){  
    // L是采用静态链表表示的顺序表，对L进行基数排序，  
    //使得L成为按关键字从小到大的有序静态链表，  
    //L.r[0]为头节点  
    for (i=0; i<L.recnum; ++i) L.r[i].next=i+1;  
    Next[L.recnum] = 0;  
    // 将L视作静态(循环)链表  
    for (i=0;i<L. keynum-1; ++i) {  
        // 按最低位优先依次对各关键字进行分配和收集  
        Distribute(L.r, i, f, e); // 第 k 趟分配  
        Collect(L.r, i, f, e);    // 第 k 趟收集  
    }// for  
} // LRadixSort
```

初始状态:

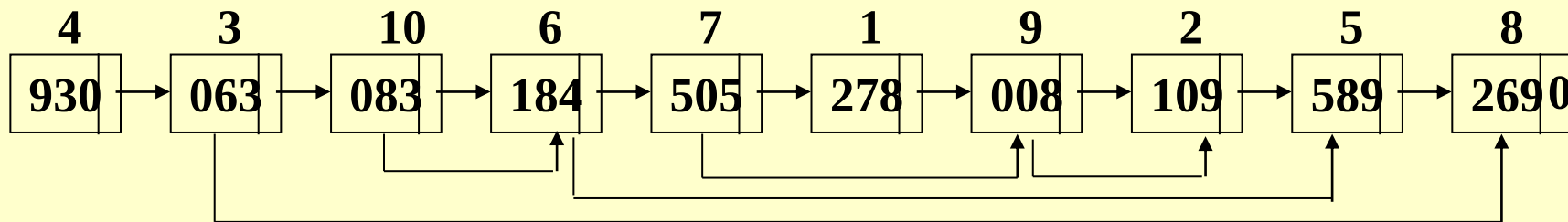


$f[0] = 4$	$e[0] = 4$
$f[1] = 0$	$e[1] =$
$f[2] = 0$	$e[2] =$
$f[3] = 3$	$e[3] = 10$
$f[4] = 6$	$e[4] = 6$
$f[5] = 7$	$e[5] = 7$
$f[6] = 0$	$e[6] =$
$f[7] = 0$	$e[7] =$
$f[8] = 1$	$e[8] = 9$
$f[9] = 2$	$e[9] = 8$

子表在分配阶段已连接好，收集阶段负责将子表串起来。

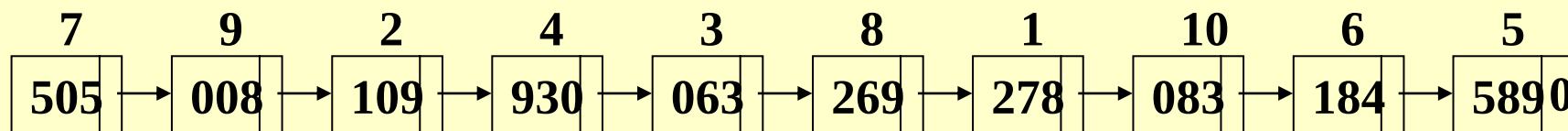


一趟收集:

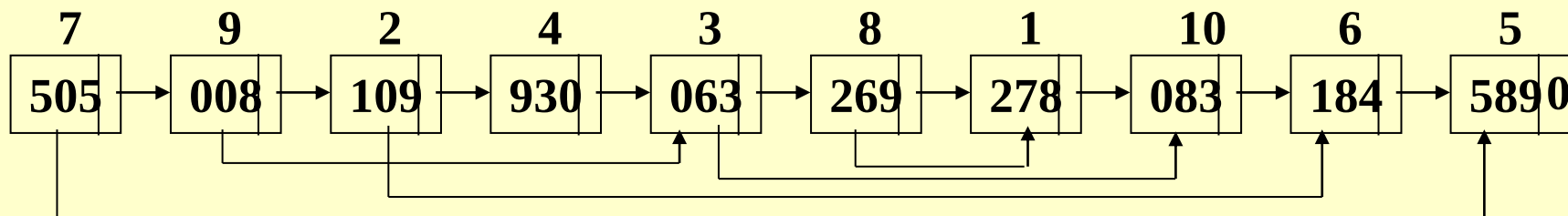


$f[0] = 7$	$e[0] = 2$
$f[1] = 0$	$e[1] =$
$f[2] = 0$	$e[2] =$
$f[3] = 4$	$e[3] = 4$
$f[4] = 0$	$e[4] =$
$f[5] = 0$	$e[5] =$
$f[6] = 3$	$e[6] = 8$
$f[7] = 1$	$e[7] = 1$
$f[8] = 10$	$e[8] = 5$
$f[9] = 0$	$e[9] =$

二趟收集:

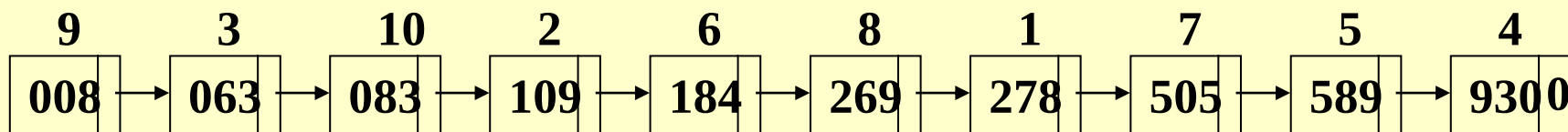


二趟收集:



$f[0] = 9$	$e[0] = 10$
$f[1] = 2$	$e[1] = 6$
$f[2] = 8$	$e[2] = 1$
$f[3] = 0$	$e[3] =$
$f[4] = 0$	$e[4] =$
$f[5] = 7$	$e[5] = 5$
$f[6] = 0$	$e[6] =$
$f[7] = 0$	$e[7] =$
$f[8] = 0$	$e[8] =$
$f[9] = 4$	$e[9] = 4$

三趟收集:



❖ 算法评价

❧ 时间复杂度:

❖ 一趟分配(n 个结点的分配) : $T(n)=O(n)$

❖ 一趟收集 (对 $radix$ 个队列的收集) :

$$T(n)=O(radix)$$

$$T(n)=O(d(n+radix))$$

其中: n ——记录数

d ——关键字数

$radix$ ——关键字取值范围 (基)

当 n 很大时: $T(n)=O(dn)$

❧ 空间复杂度: $S(n)=2radix$ 个队列指针+ n 个指针域空间

返回

各种排序方法各有优缺点，故在不同情况下可作不同的选择。通常需考虑的因素有：**待排序的记录个数；记录本身的大小；记录的键值分布情况等。**

在顺序存储结构上实现时：

- 若待排序的记录个数 n 较小时，可采用简单排序方法。
- 若 n 较大时，应采用快速排序或堆排序。
- 若待排序的记录已基本有序，可采用直接插入排序和起泡排序

表排序： 表插入排序、链式基数排序

1.按**平均的时间性能**来分，**有三类**排序方法：

时间复杂度为 $O(n\log n)$ 的方法有：**快速排序、堆排序和归并排序**，其中以快速排序为最好；

时间复杂度为 $O(n^2)$ 的有：**直接插入排序、起泡排序和简单选择排序**，其中以直接插入为最好，特别是对那些对关键字近似有序的记录序列尤为如此；

时间复杂度为 $O(dn)$ 的排序方法只有，基数排序。

2.当**待排记录序列按关键字顺序有序**时，**直接插入排序和起泡排序**能达到 $O(n)$ 的时间复杂度；而对于快速排序而言，这是最不好的情况，此时的时间性能蜕化为 $O(n^2)$ ，因此是应该尽量避免的情况。

3.简单选择排序、堆排序和归并排序的时间性能不随记录序列中关键字的分布而改变。

从空间性能来看

指的是排序过程中所需的辅助空间大小。

1. 所有的简单排序方法(包括：直接插入、起泡和简单选择)和堆排序的空间复杂度为 $O(1)$;
2. 快速排序为 $O(\log_2 n)$ ，为栈所需的辅助空间;
3. 归并排序所需辅助空间最多，其空间复杂度为 $O(n)$;
4. 链式基数排序需附设队列首尾指针，则空间复杂度为 $O(rd)$ 。

从排序方法的稳定性能

稳定的排序方法的判定：对于两个关键字相等的记录，它们在序列中的相对位置，在排序之前和经过排序之后，没有改变。

当对多关键字的记录序列进行LSD方法排序时，必须采用稳定的排序方法。

快速排序和堆排序是不稳定的排序方法。

例如：

快速排序	{ <u>512</u> 275 275* }	排序前
	{ 275* 275 512 }	排序后

返回

算法种类	时间复杂度			空间复杂度	是否稳定
	最好情况	平均情况	最坏情况		
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否
希尔排序				$O(1)$	否
快速排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	否
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	否
2 路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	是
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是

练习与作业

- 本次课练习：
- 本次课作业：习题10 1（5）（6）

返回

- 40、以下排序方法中，（）在一趟结束后不一定能选出一个元素放在其最终位置上。
A、简单选择排序 B、冒泡排序 C、归并排序 D、堆排序
- 41、以下排序方法中，（）不需要进行关键字的比较。
A、快速排序 B、基数排序 C、归并排序 D、堆排序
- 42、下列4种排序方法中，排序过程中的比较次数与序列初始状态无关的是（）。
A、归并排序 B、插入排序 C、快速排序 D、冒泡排序
- 43、若对27个元素只进行三趟多路归并排序，则选取的归并路数最少为（）。
A、2 B、3 C、4 D、5
- 44、【2016统考真题】对10TB的数据文件进行排序，应使用的方法是（）。
A、希尔排序 B、堆排序 C、快速排序 D、归并排序
- 45、2路归并排序中，归并趟数的数量级是（）。
A、 $O(n)$ B、 $O(\log_2 n)$ C、 $O(n \log_2 n)$ D、 $O(n^2)$
- 46、一组经过第一趟2路归并排序后的记录的关键字为{25,50,15,35,80,85,20,40,36,70}，其中包含5个长度为2的有序表，用2路归并排序方法对该序列进行第二趟归并后的结果为（）。
A、15,25,35,50,80,20,85,40,70,36 B、15,25,35,50,20,40,80,85,36,70
C、15,25,50,35,80,85,20,36,40,70 D、15,25,35,50,80,20,36,40,70,85
- 47、对{05,46,13,55,94,17,42}进行基数排序，一趟排序的结果是（）。
A、05,46,13,55,94,17,42 B、05,13,17,42,46,55,94
C、42,13,94,05,55,46,17 D、05,13,46,55,17,42,94
- 48、【2013统考真题】对给定的关键字序列{110,119,007,911,114,120,122}进行基数排序，第2趟分配收集后得到的关键字序列是（）。
A、007,110,119,114,911,120,122 B、007,110,119,114,911,122,120
C、007,110,911,114,119,120,122 D、110,120,911,122,114,007,119
- 49、若要求排序是稳定的，且关键字为实数，则在下列排序方法中应选（）。
A、直接插入排序 B、选择排序 C、基数排序 D、快速排序